

AnyCam 2.2.8-rc2.4 — Audit Report

Build date: 2026-04-30

Source baseline: 2.2.8-rc2.3

Scope: Two-bug bugfix release. Empirically validated via the 2026-04-30 11:34 rc2.3 log on the live install — both bugs visible in the same session.

What rc2.4 changes

Two narrowly-scoped bug fixes. No other functional changes. No schema, version, or CAMERA_DB changes. No build / dependency churn.

Fix 1 — Manual tier change silently no-ops (primary)

Symptom (reproduced live in rc2.3 log at 11:55:38-11:59:01):

User had previously set a manual tier (profile[1] / 720p) on the .22 Microseven. Closed Enhanced view, reopened it after ~13 minutes idle. Server correctly resumed at preserved tier 31 (profile[1] 720p) — log shows `adaptive focus — switching to profile[1] for tier 31` and `ffmpeg launch at 1280x720`.

User then picked 4K (`tier [0] profile[0]`) from the Resolution dropdown. Log shows the manual-tier line:

```
11:55:49 Focus [...]: manual tier [0] profile[0] fps=None
```

But no `killed ffmpeg to apply manual tier change` line followed.

Frame 100 onwards stayed at ~360KB (the 720p signature, 4K is ~2MB). `ffmpeg` never restarted. Stream stuck at 720p for the next 4 minutes.

Root cause (race condition):

`snap_loop`'s outer `finally` block contained:

```
finally:
    state["proc"] = None
    if state.get("task") is asyncio.current_task():
        state["task"] = None
```

The `state["task"]` clear was correctly gated on `asyncio.current_task()` ownership — that gate was added in an earlier release to fix the analogous duplicate-loop bug where a winding-down task would clobber a successor's task reference.

The same gate was missing for `state["proc"]`. Sequence of events that triggered the bug at 11:55:38:

1. Background `snap_loop` (TASK_A) running 480px sub-stream from 11:52:42. `state["proc"]` = TASK_A's `ffmpeg`.
2. User opens Enhanced view → `handle_focus_set` cancels TASK_A, creates TASK_B (native_res `snap_loop`).
3. TASK_B runs, hits `proc = await _launch_snap()`, sets `state["proc"]` = TASK_B's `ffmpeg` (line 4697).
4. TASK_A's `CancelledError` fires through inner handler → outer `finally` runs → `state["proc"] = None` clobbers TASK_B's reference.
5. TASK_B continues reading frames — but `state["proc"]` is now `None` even though `ffmpeg` is alive.
6. User picks 4K → `handle_focus_set_tier` reads `state.get("proc")`

→ `None` → kill path skipped → ada updated with new tier_idx but ffmpeg never restarted at the new profile.

Why this didn't manifest on the FIRST manual tier change in the session: the first time, `state["proc"]` was set BEFORE any task cancellation. The race only triggers when a fresh enter-Enhanced-view cancels an existing background task.

Fix: apply the same conditional-clear pattern that already protected `state["task"]` :

```
finally:
    if state.get("task") is asyncio.current_task():
        state["proc"] = None
        state["task"] = None
```

A successor task's `state["proc"]` write now survives the predecessor's cleanup. `handle_focus_set_tier`'s kill path fires correctly on every manual change, including post-re-entry.

Fix 2 — 4K-fallback toast fires on manual changes

Symptom:

When the user picked a non-4K resolution from the dropdown, the toast “4K too demanding for this hardware — falling back to secondary stream” appeared. The user's hypothesis was that this toast was *causing* the stuck-at-720p behavior (Fix 1) by locking the lower tier. It wasn't — the toast was a coincidental separate bug — but the toast itself was also wrong: it implied auto-degrade when the user was driving.

Root cause:

The toast fires in the `_startFocusPoll` poll handler whenever the `X-Step-Res` response header changes from a 4K-class resolution (≥ 3840 wide) to anything smaller. The check did not distinguish auto-degrade (server-driven) from manual change (user-driven) — both produce the same `X-Step-Res` transition.

Fix: gate the toast on `!_manualTierActive`. The `_manualTierActive` flag is already set to `true` by `focusPickRes` / `focusPickFps` (and by rc2.3's `_loadFocusProfiles` when the server returns `manual_override: true`), and reset to `false` by `focusResetAuto`. Genuine auto-degrade still surfaces the toast as before; user-driven changes don't.

What didn't change

- No changes to the adaptive controller's `fast_death` / `restart_overflow` decision logic. `manual_override` already correctly skipped these checks (lines 4960, 4976) — the apparent “locking at lower tier despite manual override” the user observed was the proc-race in Fix 1, not the adaptive logic ignoring the override flag.
- `handle_focus_set_tier` itself is unchanged. The kill path was correctly written; the bug was upstream in `state["proc"]` being nulled before the kill executed.
- No CAMERA_DB changes, no schema changes, no `verify_release.py` contract changes.

Verification

`verify_release.py` passes all five gates:

Gate	Result

<code>ast.parse()</code>	✓
125 top-level definitions, no duplicates	✓
Semantic contract checks (12 function contracts)	✓
rc2 CAMERA_DB throttle field validation	✓
Best-practice audit	✓
Version consistency (config.yaml + Python constant)	✓ at 2.2.8-rc2.4
CHANGELOG.md top entry matches version	✓

Test plan for the live install

Fix 1 verification — proc race / manual switch back to 4K:

1. Install rc2.4. Trigger a fresh scan. Authenticate the .22.
2. Open Enhanced view. Pick 720p from Resolution dropdown. Confirm stream switches.
3. Close Enhanced view (X button).
4. Wait 30+ seconds for the background snap_loop to start (or do nothing — re-entry will cancel whatever is running).
5. Re-open Enhanced view. Server should resume at 720p (preserved).
6. Pick 4K from Resolution dropdown.
7. **Expected (rc2.4):** log shows `Focus [...]: manual tier [0] profile[0] fps=None` IMMEDIATELY followed by `Focus [...]: killed ffmpeg to apply manual tier change`. Stream switches to 4K within ~6 seconds (ffmpeg startup time).
8. **Previous (rc2.3):** the `killed ffmpeg` line was missing, stream stayed at 720p.

(Note: the .22's 4K HEVC stream itself is still software-decode unstable — ffmpeg will keep EOFing every ~30 frames at 4K. That's a camera/decoder issue separate from this fix and expected to improve with rc3 hardware decode work. But the *switch* should now happen immediately.)

Fix 2 verification — toast suppression:

1. In Enhanced view on the .22, pick 720p from the Resolution dropdown while ffmpeg is currently running 4K.
2. **Expected (rc2.4):** no toast appears. Stream switches to 720p.
3. **Previous (rc2.3):** toast popup appeared: "4K too demanding for this hardware — falling back to secondary stream"

The genuine auto-degrade path (where the system, not you, picks the lower tier) should still show the toast — but on the .22 currently that won't happen because rc2.4's Fix 1 properly respects manual override and the `manual_override` skip in the adaptive controller prevents auto-stepping. The toast logic is correct, just better gated.

Known latent issues carried forward (unchanged)

- **Dropdown rapid-flicker bug** — poll loop at lines ~7510-7526 unconditionally sets `sel.disabled`, `sel.title`, and parent `.focus-ctrl-group` opacity every 60ms based on `snapMode`. When the camera oscillates between RTSP and http_snap fallback, this rapidly toggles disabled/enabled and closes any open dropdown menu. **Not fixed in rc2.4 at user's explicit request** — flagged

for rc3+ alongside dtoverlay / hardware-decode work.

- “Unknown child process pid X” asyncio warning — cosmetic, Python 3.8–3.10 child-watcher race. Future cleanup:
`asyncio.set_child_watcher(asyncio.PidfdChildWatcher())` at startup (Python 3.9+ Linux).
- .22 4K HEVC software-decode instability — camera/decoder cost issue. Expected to improve with rc3 hardware decode.
- `_estimateCpuPct` and `_focusWarnOK` orphaned dead JS (since 2.2.7) — defer to next maintenance cycle.
- Pi 4 dtoverlay auto-toggle flow — deferred to rc3.
- Credential leak: `stream_profiles` array in API response contains embedded credentials in URLs.

End of audit report — AnyCam 2.2.8-rc2.4